HDOC – DAY 4

C Programming

Multiplication of Two Integers Without Using the * Operator

1. Mistakes in the Given Algorithm

- It always adds num1 repeatedly, so the number of additions depends on num2. This is not the minimum possible number of additions.
- It assumes num2 is suitable as a positive loop count. A negative num2 can make the repetition condition fail immediately.
- It does not correctly handle negative × positive, positive × negative, and negative × negative cases.
- It does not explicitly handle multiplication by zero and therefore does not guarantee 0 additions for zero cases.
- It does not choose the smaller absolute value as the repetition count, so cases such as 3 × 12 use 12 additions instead of the required 3.

2. Corrected Algorithm

1. Define num1, num2, a, b, prod, count, sign.

2. Get num1, num2.

3. If num1 = 0 OR num2 = 0

   - Set prod = 0
   - Print prod
   - Stop

4. Set sign = 1

5. If num < 0

   - sign = -sign
   - num1 = -num1

6. If num2 < 0

   - sign = -sign
   - num2 = -num2

7. If num1 > num2

- Swap num1 and num2

8. Set prod = 0, count = 1

9. Repeat until count > num1

- prod = prod + num2
- count = count + 1

10. If sign = -1

- prod = -prod

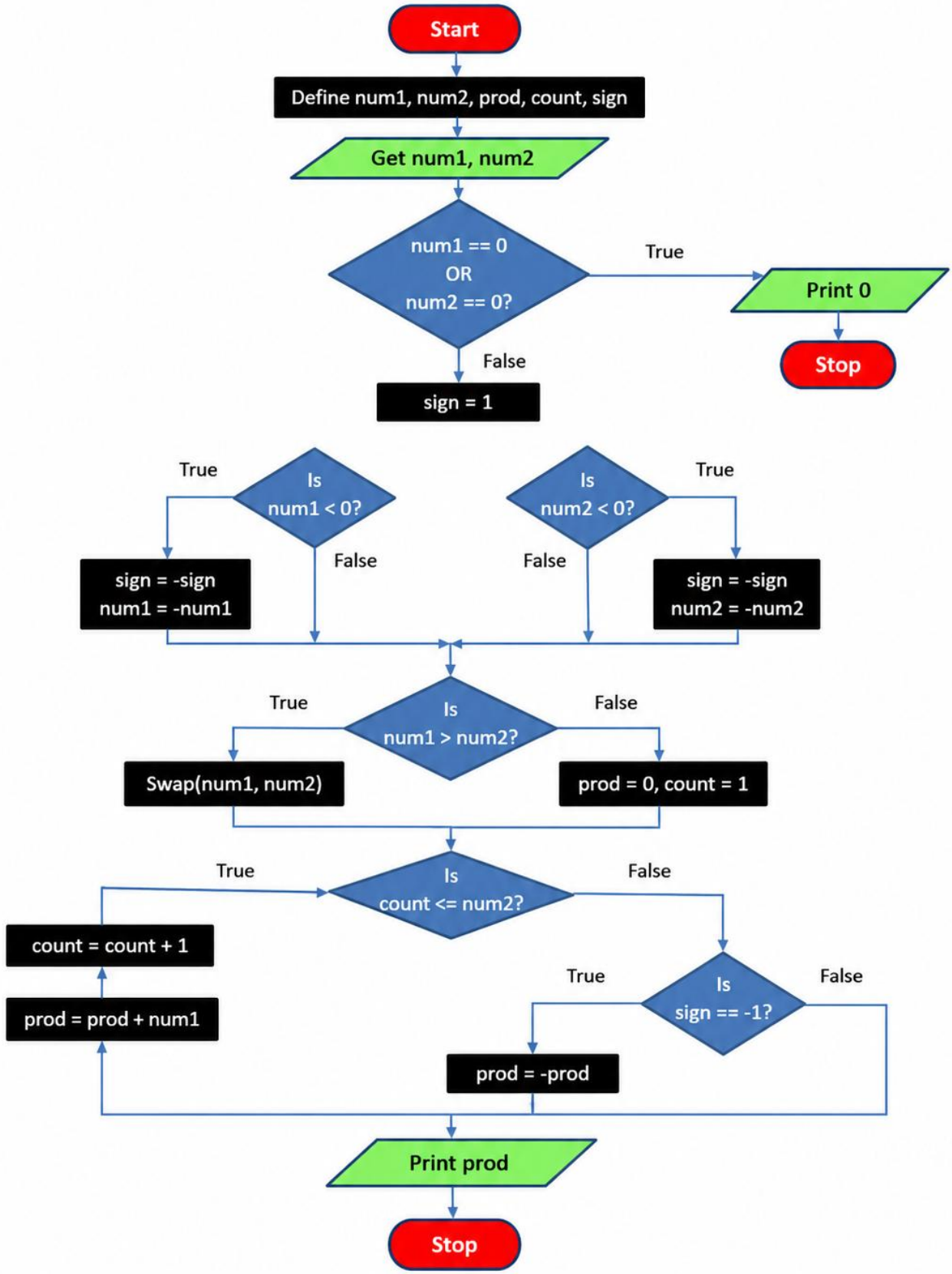
11. print prod

12. stop

3. Mistakes in the Given Flowchart

- The flowchart repeats the same incorrect logic as the algorithm: prod = prod + num1 is performed num2 times.
- There is no step for converting negative inputs to absolute values before repetition.
- There is no sign-handling decision for the four combinations of positive and negative integers.
- There is no proper zero check to guarantee 0 additions when either input is zero.
- The flowchart does not choose the smaller absolute value as the repetition count, so it fails the minimum-additions requirement.
- The loop condition is unsuitable for negative values of num2.

4. Corrected Flowchart


Start
Define num1, num2, prod, count, sign
Get num1, num2
num1 == 0
OR
num2 == 0?
True
Print 0
Stop
False
sign = 1
Is
num1 < 0?
True
False
sign = -sign
num1 = -num1
Is
num2 < 0?
True
False
sign = -sign
num2 = -num2
Is
num1 > num2?
True
False
Swap(num1, num2)
prod = 0, count = 1
Is
count <= num2?
True
False
count = count + 1
prod = prod + num1
Is
sign == -1?
True
False
prod = -prod
Print prod
Stop

5. Verification Using the Given Test Cases

Test Case 1: 12 × 3

Input: num1 = 12, num2 = 3

Both numbers are positive, so sign = +1. Since 12 > 3, swap them: num1 = 3, num2 = 12.

Set prod = 0. Add num2 three times because the smaller absolute value is 3:

prod = 0 + 12 = 12

prod = 12 + 12 = 24

prod = 24 + 12 = 36

Number of additions = 3. Final product = 36. Result: Pass.

Test Case 2: 3 × 12

Input: num1 = 3, num2 = 12

Both numbers are positive, so sign = +1. Since 3 < 12, no swap is required.

Set prod = 0. Add num2 three times:

prod = 0 + 12 = 12

prod = 12 + 12 = 24

prod = 24 + 12 = 36

Number of additions = 3. Final product = 36. Result: Pass.

Test Case 3: -6 × 5

Input: num1 = -6, num2 = 5

num1 is negative, so sign = -1 and num1 becomes 6. num2 is positive. Since 6 > 5, swap them: num1 = 5, num2 = 6.

Set prod = 0. Add 6 five times:

prod = 0 + 6 = 6

prod = 6 + 6 = 12

prod = 12 + 6 = 18

prod = 18 + 6 = 24

prod = 24 + 6 = 30

Number of additions = 5. Apply sign = -1, so final product = -30. Result: Pass.

Test Case 4: 6 × -5

Input: num1 = 6, num2 = -5

num1 is positive. num2 is negative, so sign = -1 and num2 becomes 5. Since 6 > 5, swap them: num1 = 5, num2 = 6.

Set prod = 0. Add 6 five times:

prod = 0 + 6 = 6

prod = 6 + 6 = 12

prod = 12 + 6 = 18

prod = 18 + 6 = 24

prod = 24 + 6 = 30

Number of additions = 5. Apply sign = -1, so final product = -30. Result: Pass.

Test Case 5: -9 × -4

Input: num1 = -9, num2 = -4

Both numbers are negative. num1 changes to 9 and flips sign to -1; num2 changes to 4 and flips sign back to +1. Since 9 > 4, swap them: num1 = 4, num2 = 9.

Set prod = 0. Add 9 four times:

prod = 0 + 9 = 9

prod = 9 + 9 = 18

prod = 18 + 9 = 27

prod = 27 + 9 = 36

Number of additions = 4. Final product = 36. Result: Pass.

Test Case 6: -4 × -9

Input: num1 = -4, num2 = -9

Both numbers are negative. num1 changes to 4 and flips sign to -1; num2 changes to 9 and flips sign back to +1. Since 4 < 9, no swap is required.

Set prod = 0. Add 9 four times:

prod = 0 + 9 = 9

prod = 9 + 9 = 18

prod = 18 + 9 = 27

prod = 27 + 9 = 36

Number of additions = 4. Final product = 36. Result: Pass.

Test Case 7: 0 × 8

Input: num1 = 0, num2 = 8

Since num1 = 0, the zero check is true. Set prod = 0 and stop.

Number of additions = 0. Final product = 0. Result: Pass.

Test Case 8: 8 × 0

Input: num1 = 8, num2 = 0

Since num2 = 0, the zero check is true. Set prod = 0 and stop.

Number of additions = 0. Final product = 0. Result: Pass.

Test Case 9: 0 × -7

Input: num1 = 0, num2 = -7

Since num1 = 0, the zero check is true before sign handling. Set prod = 0 and stop.

Number of additions = 0. Final product = 0. Result: Pass.

Test Case 10: -7 × 0

Input: num1 = -7, num2 = 0

Since num2 = 0, the zero check is true before sign handling. Set prod = 0 and stop.

Number of additions = 0. Final product = 0. Result: Pass.

Test Case 11: 0 × 0

Input: num1 = 0, num2 = 0

Since both inputs are zero, the zero check is true. Set prod = 0 and stop.

Number of additions = 0. Final product = 0. Result: Pass.

Test Case 12: 1 × -7

Input: num1 = 1, num2 = -7

num1 is positive. num2 is negative, so sign = -1 and num2 becomes 7. Since 1 < 7, no swap is required.

Set prod = 0. Add 7 once:

prod = 0 + 7 = 7

Number of additions = 1. Apply sign = -1, so final product = -7. Result: Pass.

Test Case 13: -1 × -9

Input: num1 = -1, num2 = -9

Both numbers are negative. num1 changes to 1 and flips sign to -1; num2 changes to 9 and flips sign back to +1. Since 1 < 9, no swap is required.

Set prod = 0. Add 9 once:

prod = 0 + 9 = 9

Number of additions = 1. Final product = 9. Result: Pass.

Test	Input	Expected Output	Additions	Result
1	12 × 3	Product: 36	3	Pass
2	3 × 12	Product: 36	3	Pass
3	-6 × 5	Product: -30	5	Pass
4	6 × -5	Product: -30	5	Pass
5	-9 × -4	Product: 36	4	Pass
6	-4 × -9	Product: 36	4	Pass
7	0 × 8	Product: 0	0	Pass
8	8 × 0	Product: 0	0	Pass
9	0 × -7	Product: 0	0	Pass
10	-7 × 0	Product: 0	0	Pass

11	0 × 0	Product: 0	0	Pass
12	1 × -7	Product: -7	1	Pass
13	-1 × -9	Product: 9	1	Pass

****